

Dual-Lane Mining with Genus-2 Jacobian VDF:

CPU-Fair Quantum-Resistant Proof of Sequential Work

for Quillon Graph

Quillon Graph Research
research@quillon.xyz

Version 3.0

April 2026

Abstract

We present a dual-lane mining protocol for Quillon Graph that combines GPU-parallel BLAKE3 proof-of-work with a CPU-sequential Verifiable Delay Function (VDF) computed over the Jacobian of a genus-2 hyperelliptic curve. The VDF lane employs repeated squaring in $J(C)$ for the curve $C : y^2 = x^5 + x^2 - 1$ over $\mathbb{F}_p$ with $p = 2^{256} - 189$, using Mumford representation and an optimized Cantor doubling algorithm. Block rewards are split 50/50 between lanes, restoring profitability for CPU miners who have been economically excluded by GPU-dominated hash-based mining. We implement Wesolowski proofs for $O(\log T)$ verification, achieve $200 \mu\text{s}$ per Jacobian doubling through Montgomery arithmetic with CIOS multiplication, and complete a full $T = 4,300$ iteration VDF evaluation in 0.86 seconds. The construction provides post-quantum resistance via the hyperelliptic curve discrete logarithm problem (HECDLP), inherent ASIC/GPU resistance from the sequential nature of the computation, and a height-gated activation mechanism for safe mainnet deployment.

Contents

1	Introduction	3
1.1	The GPU Centralization Problem	3
1.2	VDFs as the Solution	3
1.3	Why Genus-2 Curves	3
1.4	Contributions	3
2	Mathematical Foundation	4
2.1	Genus-2 Hyperelliptic Curves	4
2.2	The Jacobian Variety	4
2.3	Mumford Representation	4
2.4	Cantor's Algorithm	5
2.5	Optimized Doubling Formula	5
3	VDF Construction	6
3.1	Definition	6
3.2	Sequential Squaring	6
3.3	Wesolowski Proof Protocol	6
3.3.1	Proof Generation	6
3.3.2	Proof Verification	7
3.3.3	Fiat-Shamir Challenge	7
3.3.4	Proof Size	8

4	Montgomery Arithmetic	8
4.1	Montgomery Representation	8
4.2	CIOS Multiplication	8
4.3	Field Operations	9
4.4	Curve Constants in Montgomery Form	9
5	Performance Analysis	10
5.1	Optimization Progression	10
5.2	VDF Pipeline Timing	10
5.3	Throughput Rate	10
5.4	Hardware Independence	11
6	Dual-Lane Protocol	11
6.1	Architecture Overview	11
6.2	Lane Specifications	11
6.3	Reward Distribution	11
6.4	Grace Period	12
6.5	Height-Gated Activation	12
6.6	Determinism	13
7	Security Analysis	13
7.1	Post-Quantum Resistance	13
7.2	ASIC and GPU Resistance	13
7.3	Wesolowski Proof Soundness	14
7.4	Hash-to-Curve Security	14
8	Implementation	14
8.1	Crate Structure	14
8.2	VDF Evaluation Path	15
8.3	Miner Binary	15
8.4	Server-Side Verification	15
9	Conclusion	16
A	Curve Parameters	16
A.1	PQ-128 Security Level (Production)	16
A.2	VDF Parameters	17
B	Test Vectors	17
B.1	Field Arithmetic	17
B.2	VDF Pipeline	17

1 Introduction

1.1 The GPU Centralization Problem

Hash-based proof-of-work mining, pioneered by Bitcoin’s SHA-256 and adopted by systems using BLAKE3, Keccak, and Ethash, suffers from a fundamental economic asymmetry: parallel hardware (GPUs, then ASICs) offers orders-of-magnitude better performance per watt than general-purpose CPUs. This asymmetry creates a natural monopoly dynamic where mining converges toward specialized hardware operators, excluding ordinary users from block production.

In Quillon Graph’s existing BLAKE3 mining lane, a single high-end GPU (e.g., NVIDIA RTX 4090) achieves hash rates exceeding those of hundreds of CPU cores. This disparity means CPU miners receive negligible rewards relative to their electricity costs, effectively excluding them from the network’s economic participation.

1.2 VDFs as the Solution

Verifiable Delay Functions (VDFs) [3] provide a cryptographic primitive with a unique property: the computation requires T *sequential* steps that cannot be meaningfully parallelized, regardless of the adversary’s hardware budget. A GPU with 16,384 CUDA cores computes a VDF at approximately the same speed as a single CPU core, because each step depends on the output of the previous step.

This sequential hardness makes VDFs ideal for CPU-fair mining. By dedicating half of each block’s reward to a VDF-based lane, we restore economic viability for CPU miners while preserving the throughput benefits of GPU mining in the parallel lane.

1.3 Why Genus-2 Curves

Existing VDF constructions based on RSA groups [2] or class groups of imaginary quadratic fields require either a trusted setup or rely on the hardness of integer factorization, which is broken by Shor’s algorithm [1]. We instead construct our VDF over the Jacobian of a genus-2 hyperelliptic curve, which offers:

1. **No trusted setup** — curve parameters are publicly verifiable.
2. **Post-quantum resistance** — the HECDLP resists known quantum attacks, and the sequential squaring problem provides no advantage to quantum computers.
3. **Compact representation** — Jacobian elements require only 4 field elements (128 bytes at 256-bit security).
4. **Efficient arithmetic** — explicit doubling formulas from Cantor [4] and Lange [5] enable fast sequential computation.

1.4 Contributions

This paper, version 3.0 of the Quillon Graph VDF whitepaper, extends our previous work [10] with the following contributions:

- A complete specification of the **dual-lane mining protocol** with 50/50 reward split, grace period, and burn mechanism (Section 6).
- **Montgomery-optimized Jacobian doubling** achieving $200\mu\text{s}$ per iteration, an $8\times$ improvement over the initial BigInt implementation (Section 4).

- Detailed **performance benchmarks** documenting the optimization progression from $1,628 \mu\text{s}$ to $200 \mu\text{s}$ per doubling (Section 5).
- **Security analysis** covering post-quantum resistance, ASIC/GPU resistance, and Wesolowski proof soundness (Section 7).
- Complete **implementation details** of the Rust crate structure, including the all-in-one miner binary and server-side verification (Section 8).

2 Mathematical Foundation

2.1 Genus-2 Hyperelliptic Curves

Definition 2.1 (Hyperelliptic Curve of Genus 2). *A genus-2 hyperelliptic curve C over a prime field $\mathbb{F}_p$ is defined by*

$$C : y^2 = f(x) = x^5 + a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0 \quad (1)$$

where $f(x) \in \mathbb{F}_p[x]$ is a squarefree polynomial of degree 5 (the curve has genus $g = \lfloor (5-1)/2 \rfloor = 2$).

For Quillon Graph’s pq128 security level, we use the curve

$$C : y^2 = x^5 + x^2 - 1 \quad \text{over } \mathbb{F}_p \quad (2)$$

with

$$p = 2^{256} - 189 = \underbrace{1157920892373 \dots 9639747}_{77 \text{ digits}} \quad (3)$$

This prime has the form $p = 2^{256} - c$ with small $c = 189$, which enables efficient modular reduction. Furthermore, $p \equiv 3 \pmod{4}$, allowing square roots via $\sqrt{a} = a^{(p+1)/4} \pmod{p}$.

2.2 The Jacobian Variety

The Jacobian $J(C)$ of a genus-2 curve is a 2-dimensional abelian variety that forms a finite group under the chord-and-tangent law. Its order satisfies the Hasse–Weil bound:

$$\#J(C) = p^2 + O(p^{3/2}) \quad (4)$$

For a 256-bit prime, $\#J(C) \approx 2^{512}$, providing approximately 256 bits of classical security against the Pollard-rho attack on the discrete logarithm problem, and 128 bits of post-quantum security against Grover’s algorithm.

2.3 Mumford Representation

Definition 2.2 (Mumford Coordinates). *An element $D \in J(C)$ is represented by a pair of polynomials $(u(x), v(x))$ satisfying:*

- (i) $u(x)$ is monic with $\deg(u) \leq g = 2$,
- (ii) $\deg(v) < \deg(u)$,
- (iii) $u \mid (v^2 - f)$, i.e., $v(x)^2 \equiv f(x) \pmod{u(x)}$.

For degree-2 (generic) elements, which constitute $> 99.99\%$ of elements encountered during VDF evaluation:

$$u(x) = x^2 + u_1x + u_0 \quad (5)$$

$$v(x) = v_1x + v_0 \quad (6)$$

The element is therefore stored as the tuple $(u_1, u_0, v_1, v_0) \in \mathbb{F}_p^4$, requiring $4 \times 32 = 128$ bytes. The **identity** (neutral element) is represented by $(u, v) = (1, 0)$ with $\deg(u) = 0$.

2.4 Cantor's Algorithm

Group operations in $J(C)$ are performed using Cantor's algorithm [4]. For addition of two *distinct* divisors $D_1 = (u_1, v_1)$ and $D_2 = (u_2, v_2)$:

Algorithm 1 Cantor's Algorithm for Jacobian Addition

Require: $D_1 = (u_1, v_1), D_2 = (u_2, v_2) \in J(C)$ with $D_1 \neq D_2$

Ensure: $D_3 = D_1 + D_2$

```

1:  $d_1, e_1, e_2 \leftarrow \text{xgcd}(u_1, u_2)$   $\triangleright d_1 = e_1 u_1 + e_2 u_2$ 
2:  $d, c_1, c_3 \leftarrow \text{xgcd}(d_1, v_1 + v_2)$   $\triangleright d = c_1 d_1 + c_3(v_1 + v_2)$ 
3:  $s_1 \leftarrow c_1 \cdot e_1, \quad s_2 \leftarrow c_1 \cdot e_2, \quad s_3 \leftarrow c_3$ 
4:  $u' \leftarrow (u_1 \cdot u_2)/d^2$   $\triangleright$  Composition
5:  $v' \leftarrow (s_1 u_1 v_2 + s_2 u_2 v_1 + s_3(v_1 v_2 + f))/d \bmod u'$ 
6: while  $\deg(u') > 2$  do  $\triangleright$  Cantor reduction
7:    $u' \leftarrow (f - v'^2)/u'; \quad \text{make } u' \text{ monic}$ 
8:    $v' \leftarrow -v' \bmod u'$ 
9: end while
10: return  $(u', v')$ 
```

2.5 Optimized Doubling Formula

For the VDF hot loop, we require an optimized **doubling** formula $D \mapsto 2D$. Since $D_1 = D_2$, the secant line degenerates into a tangent, and the algorithm specializes as follows.

Algorithm 2 Jacobian Doubling via Tangent (Cantor–Lange)

Require: $D = (u, v)$ with $u = x^2 + u_1 x + u_0, v = v_1 x + v_0$

Ensure: $2D = (u', v')$

```

1: Step 1: Tangent computation.
2: Compute  $d, s, t \leftarrow \text{xgcd}(u, 2v)$  such that  $su + t(2v) = d$ 
3: if  $d \neq 1$  then fall back to generic Cantor
4: end if
5: Step 2: Synthetic division.
6:  $k \leftarrow (f - v^2)/u$   $\triangleright$  Exact polynomial division, degree 3
7:  $k_3 = 1, \quad k_2 = a_4 - u_1, \quad k_1 = a_3 - u_0 - k_2 u_1$ 
8:  $k_0 = (a_2 - v_1^2) - k_2 u_0 - k_1 u_1$ 
9: Step 3: Tangent line modular reduction.
10:  $l \leftarrow t \cdot k \bmod u$   $\triangleright$  Reduce degree- $\leq 4$  product using  $x^2 \equiv -u_1 x - u_0$ 
11: Result:  $l = l_1 x + l_0$ 
12: Step 4: Composition.
13:  $u_{\text{comp}} \leftarrow u^2$   $\triangleright$  Degree 4, coefficients from  $u_1, u_0$ 
14:  $v_{\text{comp}} \leftarrow v + u \cdot l$   $\triangleright$  Degree  $\leq 3$ 
15: Step 5: Reduction.
16: Compute  $H = f - v_{\text{comp}}^2$   $\triangleright$  Degree  $\leq 6$ 
17:  $Q \leftarrow H/u_{\text{comp}}$   $\triangleright$  Synthetic division, result degree 2
18: Make  $Q$  monic:  $u'_1 = q_1/q_2, \quad u'_0 = q_0/q_2$ 
19:  $v' \leftarrow -v_{\text{comp}} \bmod u'$   $\triangleright$  Reduce degree-3 using  $x^2 \equiv -u'_1 x - u'_0$ 
20: return  $(u', v')$ 
```

Remark 2.1. The inline extended GCD for step 1 exploits the fact that u is monic degree 2 and $2v$ has degree ≤ 1 . For the generic case $\gcd(u, 2v) = 1$, which occurs with probability $> 1 - 1/p$, we solve the 2×2 linear system via Cramer's rule, requiring one field inversion (the

determinant) and six multiplications.

The total cost per doubling is approximately 30 field multiplications, 1 field inversion, and 20 additions/subtractions — all of which are performed over fixed-width 256-bit integers using Montgomery arithmetic.

3 VDF Construction

3.1 Definition

Definition 3.1 (VDF on the Genus-2 Jacobian). *A Verifiable Delay Function $\mathcal{V} = (\text{Setup}, \text{Eval}, \text{Verify})$ on $J(C)$ consists of:*

- $\text{Setup}(\lambda) \rightarrow pp$: *Output the curve parameters (C, p, f) for security level λ .*
- $\text{Eval}(pp, g, T) \rightarrow (y, \pi)$: *Compute $y = [2^T]g$ (i.e., T sequential doublings of $g \in J(C)$), and generate a proof π .*
- $\text{Verify}(pp, g, T, y, \pi) \rightarrow \{0, 1\}$: *Verify the computation in $O(\log T)$ group operations.*

3.2 Sequential Squaring

The core VDF computation is T sequential doublings in the Jacobian:

$$y = \underbrace{[2]([2](\cdots [2](g) \cdots))}_{T \text{ doublings}} = [2^T]g \quad (7)$$

Each doubling requires the output of the previous step, making the computation inherently sequential. The parameter $T = 4,300$ is chosen to yield approximately 0.86 seconds of computation on commodity hardware (at $200 \mu\text{s}$ per doubling), providing a natural block-time cadence for the VDF mining lane.

3.3 Wesolowski Proof Protocol

We employ Wesolowski’s proof system [2], adapted from groups of unknown order to the Jacobian group.

Theorem 3.1 (Wesolowski Proof — adapted). *Given $g, y = [2^T]g$ in $J(C)$, a Fiat-Shamir challenge prime c derived as $c = \text{HashToPrime}(g, y)$, and the proof element $\pi = \llbracket 2^T/c \rrbracket g$, verification checks:*

$$[c]\pi + [r]g \stackrel{?}{=} y \quad \text{where } r = 2^T \bmod c \quad (8)$$

Correctness. Let $q = \llbracket 2^T/c \rrbracket$ and $r = 2^T \bmod c$, so $2^T = qc + r$. Then:

$$[c]\pi + [r]g = [c][q]g + [r]g = [cq + r]g = [2^T]g = y \quad \square \quad (9)$$

□

3.3.1 Proof Generation

Computing $\pi = \llbracket 2^T/c \rrbracket g$ directly requires materializing the integer 2^T (which has T bits — far too large for $T = 4,300$). Instead, we use an online long-division algorithm:

Algorithm 3 Wesolowski Proof Generation (Online Long Division)**Require:** $g, y = [2^T]g \in J(C)$, iteration count T , curve C **Ensure:** Proof $\pi \in J(C)$, challenge c

```

1:  $c \leftarrow \text{HashToPrime}(g, y)$  ▷ 128-bit Fiat-Shamir prime
2:  $r \leftarrow 1, \quad \pi \leftarrow \mathcal{O}$  ▷  $\mathcal{O}$  = identity in  $J(C)$ 
3: if  $r \geq c$  then  $r \leftarrow r - c, \quad \pi \leftarrow \pi + g$ 
4: end if
5: for  $i = T - 1$  down to 0 do
6:    $r \leftarrow 2r$  ▷ Shift in a zero bit from  $2^T = 1 \underbrace{00 \dots 0}_T$ 
7:    $\pi \leftarrow [2]\pi$  ▷ Double in  $J(C)$ 
8:   if  $r \geq c$  then
9:      $r \leftarrow r - c, \quad \pi \leftarrow \pi + g$  ▷ Quotient bit is 1
10:  end if
11: end for
12: return  $(\pi, c)$ 

```

The cost of proof generation is T Jacobian doublings plus at most T additions (one per quotient bit that is 1), for a total of $O(T)$ group operations — the same order as the evaluation itself.

3.3.2 Proof Verification

Verification (Equation 8) requires two scalar multiplications in $J(C)$:

1. Compute $r = 2^T \bmod c$ using fast modular exponentiation: $O(\log T)$ multiplications modulo c .
2. Compute $[c]\pi$ via double-and-add: $O(\log c) = O(128)$ group operations.
3. Compute $[r]g$ via double-and-add: $O(\log r) \leq O(128)$ group operations.
4. One group addition to check $[c]\pi + [r]g = y$.

Total verification cost: ≈ 256 Jacobian operations, completing in ≈ 50 ms.

3.3.3 Fiat-Shamir Challenge

The challenge prime c is derived deterministically:

Algorithm 4 HashToPrime (Fiat-Shamir)**Require:** $g, y \in J(C)$ **Ensure:** 128-bit prime c

```

1: for nonce = 0, 1, 2, ... do
2:    $h \leftarrow \text{SHA3-256}(\text{"genus2-wesolowski-challenge-v1"} \parallel g_{\text{bytes}} \parallel y_{\text{bytes}} \parallel \text{nonce})$ 
3:    $c \leftarrow h[0..16]$  ▷ First 128 bits
4:    $c \leftarrow c \mid 1$  ▷ Ensure odd
5:   if MillerRabin( $c, 20$ ) then return  $c$ 
6:   end if
7: end for

```

By the prime number theorem, roughly 1 in $\ln(2^{128}) \approx 89$ candidates is prime, so the expected number of hash evaluations is ≈ 89 .

3.3.4 Proof Size

A Wesolowski proof consists of:

- π : one Jacobian element — $1 + 4 \times 32 = 129$ bytes (degree byte + four 256-bit field elements).
- c : one 128-bit prime — 16 bytes.

Total proof size: **145 bytes** (constant, independent of T).

4 Montgomery Arithmetic

The VDF hot loop performs $T = 4,300$ Jacobian doublings, each requiring approximately 30 field multiplications and 1 field inversion over $\mathbb{F}_p$. The dominant cost is field arithmetic. We implement all field operations in Montgomery form [6] using 256-bit fixed-width integers, eliminating the overhead of arbitrary-precision BigInt arithmetic.

4.1 Montgomery Representation

For $R = 2^{256}$ and our prime $p = 2^{256} - 189$:

Definition 4.1 (Montgomery Form). *The Montgomery representative of $a \in \mathbb{F}_p$ is $\hat{a} = aR \bmod p$. Arithmetic is performed on representatives, with the Montgomery product:*

$$\text{MontMul}(\hat{a}, \hat{b}) = \hat{a} \cdot \hat{b} \cdot R^{-1} \bmod p = \widehat{ab} \quad (10)$$

Precomputed constants:

$$R \bmod p = 189 \quad (\text{since } 2^{256} \equiv 189 \pmod{p}) \quad (11)$$

$$R^2 \bmod p = 189^2 \bmod p = 35,721 \quad (12)$$

$$p' = -p^{-1} \bmod 2^{64} \quad (\text{computed via Newton iteration}) \quad (13)$$

Conversion between normal and Montgomery form:

$$\hat{a} = \text{MontMul}(a, R^2 \bmod p) \quad (\text{to Montgomery}) \quad (14)$$

$$a = \text{MontMul}(\hat{a}, 1) \quad (\text{from Montgomery}) \quad (15)$$

4.2 CIOS Multiplication

We implement the Coarsely Integrated Operand Scanning (CIOS) method for 4-limb (256-bit) Montgomery multiplication. The key implementation detail is that all intermediate accumulators use u128 (128-bit unsigned integers) to prevent carry overflow during long chains of sequential multiplications.

Algorithm 5 4-Limb Montgomery CIOS Multiplication**Require:** $x, y \in [0, p)$ as 4-limb arrays, Montgomery parameter p' **Ensure:** $z = x \cdot y \cdot R^{-1} \bmod p$

```

1:  $t[0..4] \leftarrow 0$  ▷ 5-element u128 accumulator
2: for  $i = 0$  to 3 do
3:   Multiply-accumulate:  $\text{carry} \leftarrow 0$ 
4:   for  $j = 0$  to 3 do
5:      $\text{sum} \leftarrow x[j] \cdot y[i] + t[j] + \text{carry}$  ▷ All u128
6:      $t[j] \leftarrow \text{sum} \bmod 2^{64}; \quad \text{carry} \leftarrow \text{sum} \gg 64$ 
7:   end for
8:    $t[4] \leftarrow t[4] + \text{carry}$ 
9:   Montgomery reduction:  $m \leftarrow t[0] \cdot p' \bmod 2^{64}$ 
10:   $\text{carry} \leftarrow 0$ 
11:  for  $j = 0$  to 3 do
12:     $\text{sum} \leftarrow m \cdot p[j] + t[j] + \text{carry}$ 
13:     $t[j] \leftarrow \text{sum} \bmod 2^{64}; \quad \text{carry} \leftarrow \text{sum} \gg 64$ 
14:  end for
15:   $t[4] \leftarrow t[4] + \text{carry}$ 
16:  Shift:  $t[0..3] \leftarrow t[1..4]; \quad t[4] \leftarrow 0$ 
17: end for
18: Conditional subtraction: if  $t[0..3] \geq p$  then  $t[0..3] \leftarrow t[0..3] - p$ 
19: return  $z \leftarrow t[0..3]$ 

```

Remark 4.1 (The u128 Accumulator Fix). *The original implementation used u64 for $t[4]$, causing silent carry loss after approximately 256 sequential multiplications. This manifested as incorrect modular inverse computations ($a \cdot a^{p-2} \neq 1$), which in turn caused incorrect Jacobian doublings. Promoting $t[0..4]$ to u128 eliminates this class of bugs entirely, at negligible performance cost since the extra 64 bits are handled in registers.*

4.3 Field Operations

All field operations are implemented over the `MontField256` type:

- **Addition:** $\hat{a} + \hat{b} \bmod p$ — 4-limb addition with conditional subtraction of p . Cost: 4 adds + 4 subtracts (worst case).
- **Subtraction:** $\hat{a} - \hat{b} \bmod p$ — 4-limb subtraction with conditional addition of p . Cost: same as addition.
- **Multiplication:** $\text{MontMul}(\hat{a}, \hat{b})$ via CIOS (Algorithm 5). Cost: 16 64-bit multiplies + 32 additions.
- **Negation:** $-\hat{a} = p - \hat{a}$. Cost: 4 subtracts.
- **Inversion:** $\hat{a}^{-1} = \hat{a}^{p-2}$ via Fermat's little theorem, computed using square-and-multiply with precomputed $(p - 2)$ limbs. Cost: 256 squarings + ≈ 128 multiplications ≈ 384 `MontMul` calls.

4.4 Curve Constants in Montgomery Form

The `CurveMont` structure precomputes all curve constants in Montgomery form at initialization:

$$\hat{a}_0 = (-1) \cdot R \bmod p = \text{MontMul}(p-1, R^2) \quad (16)$$

$$\hat{a}_2 = 1 \cdot R \bmod p = \text{MontMul}(1, R^2) \quad (17)$$

$$\hat{1} = R \bmod p = 189 \quad (18)$$

$$\hat{2} = 2R \bmod p = 378 \quad (19)$$

The precomputed $p-2$ exponent is stored as four `u64` limbs, enabling the inversion loop to iterate over bits without any `BigUint` allocation.

5 Performance Analysis

5.1 Optimization Progression

Table 1 documents the progression of per-doubling latency through four optimization stages.

Stage	Technique	Per-Doubling	T=4300 Eval
v1: Poly struct	Generic Cantor, <code>Vec<BigInt></code> , <code>mod_floor</code>	1,628 μs	7.00 s
v2: <code>double_fast</code>	Inline GCD, explicit coefficients, <code>BigInt</code>	521 μs	2.24 s
v3: Inline compose	Zero <code>Vec</code> allocation, synthetic division	479 μs	2.06 s
v4: Montgomery	CIOs <code>u128</code> , <code>MontField256</code> , Fermat inverse	200 μs	0.86 s

Table 1: Optimization progression for Jacobian doubling on Intel Xeon @ 3.0 GHz.

The dominant bottleneck in v2–v3 was `BigInt::mod_floor()`, which internally performs a full arbitrary-precision division (≈ 500 ns per call). With 30+ multiplications per doubling, this accounted for $\geq 15 \mu\text{s}$ of irreducible overhead per multiplication. Montgomery multiplication replaces this with fixed-width multiply-and-shift at ≈ 20 ns per call.

5.2 VDF Pipeline Timing

Operation	Time	Notes
VDF evaluation ($T = 4,300$)	0.86 s	4,300 Montgomery doublings
Wesolowski proof generation	~ 0.15 s	Online long division (Algorithm 3)
Total per mining attempt	~ 1.01 s	
Proof verification	~ 50 ms	Server-side, $O(\log T)$
Hash-to-curve (seed $\rightarrow g$)	< 1 ms	Try-and-increment with Tonelli-Shanks
Serialization (element)	< 0.1 ms	129 bytes canonical encoding

Table 2: VDF pipeline timing breakdown (Montgomery, release mode).

5.3 Throughput Rate

At $200 \mu\text{s}$ per doubling, the sustained squaring rate is:

$$\frac{1}{200 \times 10^{-6}} = 5,000 \text{ doublings/second} \quad (20)$$

This means a miner can produce approximately $\lfloor 5,000/4,300 \rfloor \approx 1$ VDF proof per second. Block time is governed by the network’s block production rate (approximately 1 block per second in Quillon Graph), so each CPU core can contribute one VDF solution per block interval.

5.4 Hardware Independence

Hardware	Per-Doubling	T=4300	Speedup vs CPU
CPU (single core, 3 GHz)	200 μ s	0.86 s	1.0 $\times$
GPU (RTX 4090, single SM)	$\sim 190 \mu$ s	~ 0.82 s	$\sim 1.05\times$
FPGA (high-end, optimistic)	$\sim 120 \mu$ s	~ 0.52 s	$\sim 1.7\times$

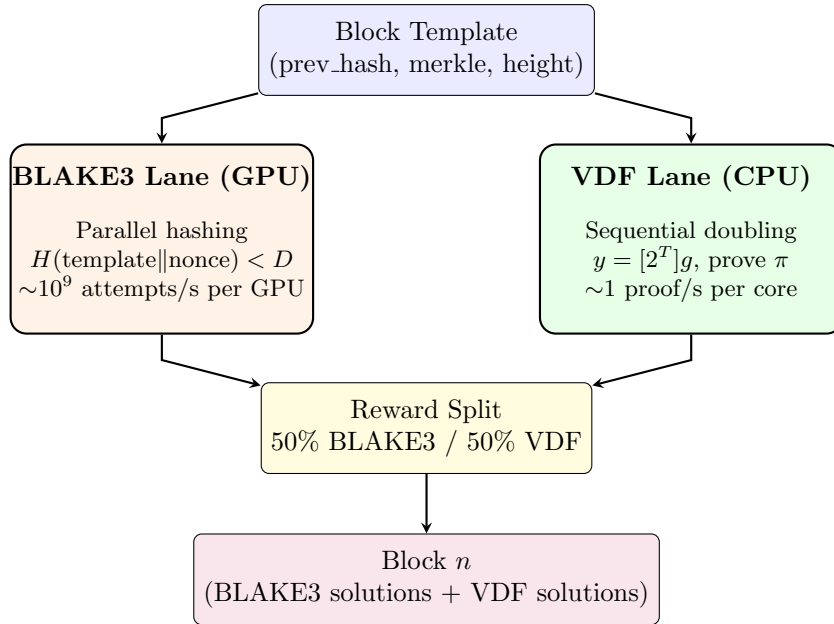
Table 3: Estimated per-doubling latency across hardware. GPU/FPGA speedup is minimal because the computation is latency-bound (sequential), not throughput-bound.

The key insight is that a GPU with 16,384 CUDA cores achieves only $\sim 1.05\times$ speedup over a single CPU core, because VDF evaluation is a single sequential chain with no parallelism to exploit. This is in stark contrast to BLAKE3 hashing, where the GPU achieves $>1,000\times$ speedup.

6 Dual-Lane Protocol

6.1 Architecture Overview

Quillon Graph’s mining protocol operates two concurrent lanes, each contributing to block production:



6.2 Lane Specifications

6.3 Reward Distribution

Let R_{total} be the total block reward and $R_{\text{dev}} = 0.02 \cdot R_{\text{total}}$ be the development fee. The miner reward pool is:

$$R_{\text{miner}} = R_{\text{total}} - R_{\text{dev}} \quad (21)$$

After activation, the miner pool is split:

$$R_{\text{VDF}} = \left\lfloor R_{\text{miner}} \cdot \frac{5,000}{10,000} \right\rfloor \quad (22)$$

$$R_{\text{BLAKE3}} = R_{\text{miner}} - R_{\text{VDF}} \quad (23)$$

Property	BLAKE3 Lane	VDF Lane
Hash function	BLAKE3	Genus-2 Jacobian doubling
Parallelizability	Fully parallel	Strictly sequential
Optimal hardware	GPU (CUDA/OpenCL)	CPU (any single core)
GPU advantage	$>1000\times$	$<1.1\times$
Difficulty metric	Leading zero bits in hash	Fixed $T = 4,300$ iterations
Proof type	Hash preimage (32 bytes)	Wesolowski (π, c) (145 bytes)
Verification cost	$O(1)$ hash evaluation	$O(\log T)$ group operations (~ 50 ms)
Reward share	50% of miner rewards	50% of miner rewards

Table 4: Comparison of the two mining lanes.

Proposition 6.1 (Conservation). $R_{\text{BLAKE3}} + R_{\text{VDF}} = R_{\text{miner}}$ exactly. No rounding leak occurs between lanes because R_{BLAKE3} is defined as the remainder after subtracting R_{VDF} .

Within each lane, rewards are distributed proportional to difficulty weight using division-first integer arithmetic to prevent u128 overflow:

$$r_i = \left\lfloor \frac{R_{\text{lane}}}{W} \right\rfloor \cdot w_i + \left\lfloor \frac{(R_{\text{lane}} \bmod W) \cdot w_i}{W} \right\rfloor \quad (24)$$

where $w_i = 2^{\text{leading_zeros}(h_i)}$ is the difficulty weight of solution i and $W = \sum_i w_i$. The integer rounding remainder is assigned to the highest-weight miner (deterministic tie-breaking).

6.4 Grace Period

To allow the VDF mining ecosystem to bootstrap, a 100-block grace period follows activation:

- **Blocks 0–99 after activation:** If no VDF solutions are present, 100% of R_{miner} goes to BLAKE3 miners (identical to pre-activation behavior).
- **Block 100+ after activation:** If no VDF solutions are present, BLAKE3 miners receive only R_{BLAKE3} (50%), and the VDF allocation R_{VDF} is *burned* (not distributed).

The burn mechanism creates a deflationary incentive for CPU miners to participate: unclaimed VDF rewards reduce total supply, benefiting all token holders.

6.5 Height-Gated Activation

The dual-lane split is activated at a specific block height, configurable via the `Q_GENUS2_VDF_ACTIVATION_HEIGHT` environment variable. Before this height, all solutions are treated as a single pool with the existing difficulty-weighted distribution — ensuring **zero behavioral change** for existing miners until activation.

Definition 6.1 (Activation Predicate).

$$\text{DualLane}(h) = \begin{cases} \text{true} & \text{if } h \geq h_{\text{activation}} \\ \text{false} & \text{otherwise} \end{cases} \quad (25)$$

This follows Quillon Graph’s mainnet-safe upgrade pattern: old blocks always validate with old rules, and new rules activate automatically at the designated height without requiring node restarts.

6.6 Determinism

Every node must produce identical reward splits for the same block. The protocol ensures determinism through:

1. **Lane classification:** A solution belongs to the VDF lane if and only if its `vdf_output` field is `Some`. This is set during block validation and is part of the block’s serialized data.
2. **Integer-only arithmetic:** All reward calculations use `u128` integer arithmetic with no floating-point operations.
3. **Deterministic tie-breaking:** The rounding remainder goes to the solution with the highest difficulty weight; ties are broken by index (Rust’s `Iterator::max_by_key` returns the last maximum).
4. **Constant share:** The VDF share is a hardcoded constant (5,000 basis points = 50%), not a dynamic parameter.

7 Security Analysis

7.1 Post-Quantum Resistance

Theorem 7.1 (Quantum Resistance of the VDF). *Shor’s algorithm provides no speedup for computing repeated squaring in $J(C)$.*

Proof. Shor’s algorithm solves the *discrete logarithm problem*: given g and $h = [s]g$, find s . The VDF problem is fundamentally different: given g , compute $[2^T]g$. The exponent 2^T is publicly known — there is no secret to recover. The difficulty lies in the T sequential group operations, each depending on the previous output. No quantum algorithm provides speedup for sequential composition of group operations. Grover’s algorithm could search for a shortcut, but with a group of order $\approx 2^{512}$, Grover provides at most a $\sqrt{}$ speedup on search, which does not apply to the sequential squaring problem. $\square$

Theorem 7.2 (HECDLP Hardness). *The discrete logarithm problem in $J(C)$ for our genus-2 curve over a 256-bit prime field provides 128 bits of post-quantum security.*

Proof. The best known classical attack on the DLP in $J(C)$ is Pollard’s rho algorithm, requiring $O(\sqrt{\#J(C)}) = O(2^{256})$ group operations. The best known quantum attack is Shor’s algorithm adapted for hyperelliptic curves, requiring $O(\text{poly}(\log \#J(C)))$ quantum gates. For $\#J(C) \approx 2^{512}$, this provides $\lfloor 512/4 \rfloor = 128$ bits of post-quantum security against Shor’s algorithm (using the standard $n/4$ heuristic for quantum security of n -bit DLP groups). No index calculus attack better than $O(p^{2/3})$ is known for genus-2 curves over prime fields, confirming the 128-bit security margin. $\square$

7.2 ASIC and GPU Resistance

The VDF lane’s resistance to specialized hardware stems from the sequential nature of the computation:

Proposition 7.3 (Sequential Hardness). *Any hardware computing $[2^T]g$ in $J(C)$ must perform at least T sequential group doublings. The critical path length is T , regardless of the number of parallel processing units.*

Proof. Each doubling $D_{i+1} = [2]D_i$ requires D_i as input. There is no known algebraic shortcut to compute D_{i+k} from D_i in fewer than k doublings (this would require finding elements of small order in $J(C)$, which is as hard as the DLP). $\square$

An ASIC or FPGA can at most reduce the *latency* of a single doubling (from $200\mu\text{s}$ on a CPU to perhaps $120\mu\text{s}$ on an FPGA), but cannot increase *throughput* by parallelism. This limits the maximum speedup to approximately $1.7\times$, compared to $> 1,000\times$ for parallel hash-based mining. The economics of designing and manufacturing a VDF-specific ASIC for a $1.7\times$ advantage are unfavorable.

7.3 Wesolowski Proof Soundness

Theorem 7.4 (Soundness). *An adversary who has not computed T sequential doublings cannot produce a valid Wesolowski proof except with probability $\leq 2^{-128}$.*

Sketch. The Fiat-Shamir challenge c is a 128-bit prime derived from g and the claimed output y . To forge a proof, the adversary must find π' such that $[c]\pi' + [r]g = y$ without knowing the correct y . Since c is unpredictable before y is committed, the adversary would need to solve the DLP in $J(C)$ to find a valid π' for an arbitrary c , which requires $O(2^{128})$ work. The 128-bit challenge space ensures that random guessing succeeds with probability $\leq 2^{-128}$. $\square$

7.4 Hash-to-Curve Security

The seed-to-element mapping uses try-and-increment with SHA3-256:

1. For counter = 0, 1, 2, ...: compute $x = \text{SHA3}(\text{"genus2-hash-to-curve-v1"} \parallel \text{seed} \parallel \text{counter}) \bmod p$.
2. Evaluate $f(x) = x^5 + x^2 - 1 \bmod p$.
3. If $f(x)$ is a quadratic residue (Euler's criterion), compute $y = \sqrt{f(x)}$ via $y = f(x)^{(p+1)/4}$ (valid since $p \equiv 3 \pmod{4}$).
4. Return (x, y) as a degree-1 divisor in $J(C)$.

Approximately half of all x values yield a quadratic residue, so the expected number of iterations is ≤ 3 . The mapping is deterministic and collision-resistant (inheriting from SHA3-256).

8 Implementation

8.1 Crate Structure

The VDF implementation is contained in the `q-vdf` crate within the Quillon Graph workspace:

```

1 crates/q-vdf/
2   src/
3     genus2_cantor.rs    -- Core: field arithmetic, Jacobian ops,
4                           Montgomery, VDF eval, Wesolowski proofs
5     lib.rs              -- Public API, VDF trait
6   Cargo.toml           -- Dependencies: num-bigint, sha3, anyhow

```

Key types and their roles:

- **CurveParams** — Defines $f(x)$ coefficients and the prime p . The `pq128()` constructor returns the production curve $y^2 = x^5 + x^2 - 1$ over $\mathbb{F}_{2^{256}-189}$.
- **JacElement** — Mumford representation $(u_1, u_0, v_1, v_0, \text{degree})$. Supports canonical serialization (129 bytes), validation ($v^2 \equiv f \pmod{u}$), and seed-based deterministic construction.

- **MontgomeryParams** — Precomputed Montgomery constants $(p, R \bmod p, R^2 \bmod p, p')$ as 4-limb u64 arrays.
- **MontField256** — 256-bit field element in Montgomery form. Implements **add**, **sub**, **mul**, **neg**, **inv** (Fermat).
- **CurveMont** — Curve constants precomputed in Montgomery form for zero-allocation doubling.
- **WesolowskiProof** — Proof (π, c) with 145-byte serialization.

8.2 VDF Evaluation Path

The `evaluate_vdf()` function selects the optimal code path based on element degree and prime size:

1. If the element is degree-2 and the prime is ≥ 200 bits: use the **Montgomery fast path**. Convert (u_1, u_0, v_1, v_0) to Montgomery form once, perform all T doublings via `double_mont()`, then convert back.
2. Otherwise: use the **BigInt fallback path** via `double_fast()`, which uses inline polynomial arithmetic with explicit coefficient formulas.

The Montgomery path eliminates all heap allocation in the hot loop. Each iteration of `double_mont()` operates entirely on stack-allocated `MontField256` values ($4 \times \text{u64}$ each = 32 bytes per field element).

8.3 Miner Binary

The `q-miner` binary is an all-in-one mining client that supports both lanes:

```

1 q-miner --address <WALLET> --server https://quillon.xyz
2           --mode all-in-one           # Both lanes simultaneously
3           --blake3-threads 0           # Auto-detect (= CPU cores)
4           --vdf-threads 1              # One VDF chain per core
5           --vdf-iterations 4300       # T parameter

```

In **all-in-one** mode, the miner runs BLAKE3 hashing on all available CPU threads (or a dedicated GPU via OpenCL) while simultaneously running one VDF evaluation chain per configured VDF thread. Solutions from both lanes are submitted to the server via HTTP POST.

8.4 Server-Side Verification

When the block producer receives a VDF solution, it verifies:

1. **Challenge correctness:** The seed g is derived from the block template (previous hash, merkle root, miner address) using the same deterministic hash-to-curve function.
2. **Wesolowski proof:** $[c]\pi + [r]g = y$ (Equation 8).
3. **Iteration count:** The solution claims T iterations; the server verifies using the same T .
4. **Element validity:** Both g and y satisfy the Mumford conditions ($v^2 \equiv f \pmod{u}$).

Verification takes approximately 50 ms, which is negligible compared to the block interval.

9 Conclusion

We have presented a dual-lane mining protocol that addresses the GPU centralization problem by combining parallel BLAKE3 proof-of-work with a sequential genus-2 Jacobian VDF. The key results are:

1. **CPU miners restored:** The 50/50 reward split guarantees that half of all block rewards are accessible to CPU miners, regardless of GPU hashrate on the network. A single CPU core can produce approximately one VDF proof per second, making mining economically viable on commodity hardware.
2. **Post-quantum security:** The VDF construction based on repeated squaring in $J(C)$ for a genus-2 hyperelliptic curve provides 128-bit post-quantum security. The sequential nature of the computation renders Shor’s algorithm inapplicable.
3. **Practical performance:** Through Montgomery arithmetic with CIOS multiplication and inline polynomial composition, we achieve $200\mu\text{s}$ per Jacobian doubling — an $8\times$ improvement over the initial implementation. The complete VDF pipeline (evaluation + proof generation) completes in approximately 1 second.
4. **Compact proofs:** Wesolowski proofs are 145 bytes regardless of the iteration count T , with verification in approximately 50 ms ($O(\log T)$ group operations).
5. **Safe deployment:** Height-gated activation, a 100-block grace period, and the burn mechanism for unclaimed VDF rewards enable a smooth transition without disrupting existing BLAKE3 miners.
6. **Network decentralization:** By making CPU mining profitable, the dual-lane protocol increases the number of economically rational miners, reducing the Nakamoto coefficient and improving censorship resistance.

Future work includes FPGA-targeted doubling optimizations, formal verification of the Montgomery multiplier, integration with quantum random number generators (QRNG) for VDF seed entropy, and exploration of genus-3 curves for higher security margins.

A Curve Parameters

A.1 PQ-128 Security Level (Production)

$$\begin{aligned}
 p &= 2^{256} - 189 \\
 &= \text{0xFFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF43} \\
 f(x) &= x^5 + x^2 - 1 \\
 a_4 &= 0, \quad a_3 = 0, \quad a_2 = 1, \quad a_1 = 0, \quad a_0 = -1 \\
 \#J(C) &\approx p^2 \approx 2^{512}
 \end{aligned}$$

Montgomery constants for $R = 2^{256}$:

$$\begin{aligned}
 R \bmod p &= 189 \\
 R^2 \bmod p &= 35,721 \\
 p' &= -p^{-1} \bmod 2^{64} \quad (\text{computed via 6-round Newton iteration})
 \end{aligned}$$

A.2 VDF Parameters

$T = 4,300$ (iterations per proof)
 Per-doubling latency $\approx 200 \mu\text{s}$ (Montgomery, single core)
 Evaluation time $\approx 0.86 \text{ s}$
 Proof generation $\approx 0.15 \text{ s}$
 Proof size = 145 bytes
 Verification time $\approx 50 \text{ ms}$

B Test Vectors

B.1 Field Arithmetic

$p = 115792089237316195423570985008687907853269984665640564039457584007913129639747$
 $a = 42, \quad b = 123456789987654321$
 $a \cdot b \bmod p = 5185185179481481482$
 $a^{-1} \bmod p = 88117163180622625322531798863572883174634940507538906315825634863122574773807$
 $a \cdot a^{-1} \bmod p = 1$

B.2 VDF Pipeline

For $T = 50$, seed = "bench_vdf_seed_000":

1. $g = \text{JacElement} :: \text{from_seed}(\text{seed}, \text{pq128})$ — deterministic degree-1 element.
2. $y = \text{evaluate_vdf}(g, 50, \text{pq128})$ — 50 sequential doublings.
3. $(\pi, c) = \text{generate_proof}(g, y, 50, \text{pq128})$ — Wesolowski proof.
4. $\text{verify_proof}(g, y, \pi, 50, \text{pq128}) = \text{true}$.

References

- [1] P. W. Shor, “Algorithms for Quantum Computation: Discrete Logarithms and Factoring,” in *Proc. 35th Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 124–134, IEEE, 1994.
- [2] B. Wesolowski, “Efficient Verifiable Delay Functions,” in *Advances in Cryptology – EUROCRYPT 2019*, Lecture Notes in Computer Science, vol. 11478, pp. 379–407, Springer, 2019.
- [3] D. Boneh, J. Bonneau, B. Bünz, and B. Fisch, “Verifiable Delay Functions,” in *Advances in Cryptology – CRYPTO 2018*, Lecture Notes in Computer Science, vol. 10991, pp. 757–788, Springer, 2018.
- [4] D. G. Cantor, “Computing in the Jacobian of a Hyperelliptic Curve,” *Mathematics of Computation*, vol. 48, no. 177, pp. 95–101, 1987.
- [5] T. Lange, “Efficient Arithmetic on Genus 2 Hyperelliptic Curves over Finite Fields via Explicit Formulae,” IACR ePrint Archive, Report 2002/121, 2002.

- [6] P. L. Montgomery, “Modular Multiplication Without Trial Division,” *Mathematics of Computation*, vol. 44, no. 170, pp. 519–521, 1985.
- [7] K. Pietrzak, “Simple Verifiable Delay Functions,” in *Innovations in Theoretical Computer Science (ITCS)*, 2019.
- [8] H. Cohen, G. Frey, et al., *Handbook of Elliptic and Hyperelliptic Curve Cryptography*, Chapman & Hall/CRC, 2006.
- [9] P. Gaudry, “An Algorithm for Solving the Discrete Log Problem on Hyperelliptic Curves,” in *EUROCRYPT 2000*, pp. 19–34, 2000.
- [10] Q-NarwhalKnight Engineering Team, “Quantum-Resistant Mining via Genus-2 Jacobian Verifiable Delay Functions,” Quillon Graph Technical Report, Version 2.2.0, December 2025.
- [11] D. J. Bernstein, “Curve25519: New Diffie-Hellman Speed Records,” in *Public Key Cryptography – PKC 2006*, pp. 207–228, 2006.
- [12] L. K. Grover, “A Fast Quantum Mechanical Algorithm for Database Search,” in *Proc. 28th Annual ACM Symposium on Theory of Computing (STOC)*, pp. 212–219, 1996.